

In [1]: `import pandas as pd`

```
df = pd.read_excel("default of credit card clients.xls")
```

In [2]: `df.shape`

Out[2]: (30001, 25)

In [3]: `df.head()`

Out[3]:

	Unnamed: 0	X1	X2	X3	X4	X5	X6	X7	X8	X9	...
--	------------	----	----	----	----	----	----	----	----	----	-----

0	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	..
---	----	-----------	-----	-----------	----------	-----	-------	-------	-------	-------	----

1	1	20000	2	2	1	24	2	2	-1	-1	..
---	---	-------	---	---	---	----	---	---	----	----	----

2	2	120000	2	2	2	26	-1	2	0	0	..
---	---	--------	---	---	---	----	----	---	---	---	----

3	3	90000	2	2	2	34	0	0	0	0	..
---	---	-------	---	---	---	----	---	---	---	---	----

4	4	50000	2	2	1	37	0	0	0	0	..
---	---	-------	---	---	---	----	---	---	---	---	----

5 rows × 25 columns



In [3]: `df.head()`

Out[3]:

	Unnamed: 0	X1	X2	X3	X4	X5	X6	X7	X8	X9	...
--	------------	----	----	----	----	----	----	----	----	----	-----

0	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	..
---	----	-----------	-----	-----------	----------	-----	-------	-------	-------	-------	----

1	1	20000	2	2	1	24	2	2	-1	-1	..
---	---	-------	---	---	---	----	---	---	----	----	----

2	2	120000	2	2	2	26	-1	2	0	0	..
---	---	--------	---	---	---	----	----	---	---	---	----

3	3	90000	2	2	2	34	0	0	0	0	..
---	---	-------	---	---	---	----	---	---	---	---	----

4	4	50000	2	2	1	37	0	0	0	0	..
---	---	-------	---	---	---	----	---	---	---	---	----

5 rows × 25 columns



In [3]: `df.head()`

Out[3]:

	Unnamed: 0	X1	X2	X3	X4	X5	X6	X7	X8	X9	...
0	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	..
1	1	20000	2	2	1	24	2	2	-1	-1	..
2	2	120000	2	2	2	26	-1	2	0	0	..
3	3	90000	2	2	2	34	0	0	0	0	..
4	4	50000	2	2	1	37	0	0	0	0	..

5 rows × 25 columns



In [4]: `import pandas as pd`

`df = pd.read_excel("default of credit card clients.xls", header=1)`

In [5]: `df.shape`

Out[5]: (30000, 25)

In [6]: `df.head()`

Out[6]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	E
0	1	20000	2	2	1	24	2	2	-1	-1	...	
1	2	120000	2	2	2	26	-1	2	0	0	...	
2	3	90000	2	2	2	34	0	0	0	0	...	
3	4	50000	2	2	1	37	0	0	0	0	...	
4	5	50000	1	2	1	57	-1	0	-1	0	...	

5 rows × 25 columns



In [7]: `df.isnull().sum()`

```
Out[7]: ID 0
        LIMIT_BAL 0
        SEX 0
        EDUCATION 0
        MARRIAGE 0
        AGE 0
        PAY_0 0
        PAY_2 0
        PAY_3 0
        PAY_4 0
        PAY_5 0
        PAY_6 0
        BILL_AMT1 0
        BILL_AMT2 0
        BILL_AMT3 0
        BILL_AMT4 0
        BILL_AMT5 0
        BILL_AMT6 0
        PAY_AMT1 0
        PAY_AMT2 0
        PAY_AMT3 0
        PAY_AMT4 0
        PAY_AMT5 0
        PAY_AMT6 0
        default payment next month 0
        dtype: int64
```

```
In [8]: df.duplicated().sum()
```

```
Out[8]: np.int64(0)
```

```
In [9]: df.duplicated().sum()
```

```
Out[9]: np.int64(0)
```

```
In [10]: df["ID"].duplicated().sum()
```

```
Out[10]: np.int64(0)
```

```
In [11]: pay_columns = ["PAY_0", "PAY_2", "PAY_3", "PAY_4", "PAY_5", "PAY_6"]

for col in pay_columns:
    print("\n", col)
    print(df[col].value_counts().sort_index())
```

```
PAY_0
PAY_0
-2      2759
-1      5686
0      14737
1       3688
2       2667
3        322
4         76
5         26
6         11
7          9
8         19
Name: count, dtype: int64
```

```
PAY_2
PAY_2
-2      3782
-1     6050
0     15730
1         28
2      3927
3       326
4        99
5         25
6         12
7         20
8          1
Name: count, dtype: int64
```

```
PAY_3
PAY_3
-2      4085
-1      5938
0     15764
1          4
2      3819
3       240
4         76
5         21
6         23
7         27
8          3
Name: count, dtype: int64
```

```
PAY_4
PAY_4
-2      4348
-1      5687
0     16455
1          2
2      3159
3       180
4         69
5         35
6          5
```

7 58

8 2

Name: count, dtype: int64

PAY_5

PAY_5

-2 4546

-1 5539

0 16947

2 2626

3 178

4 84

5 17

6 4

7 58

8 1

Name: count, dtype: int64

PAY_6

PAY_6

-2 4895

-1 5740

0 16286

2 2766

3 184

4 49

5 13

6 19

7 46

8 2

Name: count, dtype: int64

```
In [12]: df[df["PAY_0"] == -2][
          ["PAY_0", "BILL_AMT1", "PAY_AMT1", "BILL_AMT2", "PAY_AMT2"]
          ].describe()
```

```
Out[12]:
```

	PAY_0	BILL_AMT1	PAY_AMT1	BILL_AMT2	PAY_AMT2
count	2759.0	2759.000000	2759.000000	2759.000000	2.759000e+03
mean	-2.0	8793.052555	6477.920261	8610.322581	8.208022e+03
std	0.0	28224.565390	20877.338447	27288.623880	4.103935e+04
min	-2.0	-15308.000000	0.000000	-33350.000000	0.000000e+00
25%	-2.0	0.000000	0.000000	0.000000	0.000000e+00
50%	-2.0	1179.000000	1131.000000	1099.000000	1.025000e+03
75%	-2.0	5819.000000	5000.000000	5593.000000	5.047500e+03
max	-2.0	478030.000000	368199.000000	419644.000000	1.684259e+06

```
In [13]: print("EDUCATION:")
          print(df["EDUCATION"].value_counts().sort_index())
```

```

print("\nMARRIAGE:")
print(df["MARRIAGE"].value_counts().sort_index())

print("\nSEX:")
print(df["SEX"].value_counts().sort_index())

print("\nTARGET:")
print(df["default payment next month"].value_counts().sort_index())

```

EDUCATION:

EDUCATION

```

0      14
1    10585
2    14030
3     4917
4      123
5       280
6        51

```

Name: count, dtype: int64

MARRIAGE:

MARRIAGE

```

0        54
1    13659
2    15964
3       323

```

Name: count, dtype: int64

SEX:

SEX

```

1    11888
2    18112

```

Name: count, dtype: int64

TARGET:

default payment next month

```

0    23364
1     6636

```

Name: count, dtype: int64

```

In [14]: print("Education unusual values:")
print(df[df["EDUCATION"].isin([0, 5, 6])]["EDUCATION"].value_counts())

print("\nMarriage unusual values:")
print(df[df["MARRIAGE"] == 0]["MARRIAGE"].value_counts())

```

Education unusual values:

EDUCATION

5 280

6 51

0 14

Name: count, dtype: int64

Marriage unusual values:

MARRIAGE

0 54

Name: count, dtype: int64

In [15]: `df.describe().T`

Out[15]:

	count	mean	std	min	25%	50%	75%
ID	30000.0	15000.500000	8660.398374	1.0	7500.75	15000.5	22500.0
LIMIT_BAL	30000.0	167484.322667	129747.661567	10000.0	50000.00	140000.0	240000.0
SEX	30000.0	1.603733	0.489129	1.0	1.00	2.0	2.0
EDUCATION	30000.0	1.853133	0.790349	0.0	1.00	2.0	2.0
MARRIAGE	30000.0	1.551867	0.521970	0.0	1.00	2.0	2.0
AGE	30000.0	35.485500	9.217904	21.0	28.00	34.0	41.0
PAY_0	30000.0	-0.016700	1.123802	-2.0	-1.00	0.0	0.0
PAY_2	30000.0	-0.133767	1.197186	-2.0	-1.00	0.0	0.0
PAY_3	30000.0	-0.166200	1.196868	-2.0	-1.00	0.0	0.0
PAY_4	30000.0	-0.220667	1.169139	-2.0	-1.00	0.0	0.0
PAY_5	30000.0	-0.266200	1.133187	-2.0	-1.00	0.0	0.0
PAY_6	30000.0	-0.291100	1.149988	-2.0	-1.00	0.0	0.0
BILL_AMT1	30000.0	51223.330900	73635.860576	-165580.0	3558.75	22381.5	67091.0
BILL_AMT2	30000.0	49179.075167	71173.768783	-69777.0	2984.75	21200.0	64006.0
BILL_AMT3	30000.0	47013.154800	69349.387427	-157264.0	2666.25	20088.5	60164.0
BILL_AMT4	30000.0	43262.948967	64332.856134	-170000.0	2326.75	19052.0	54506.0
BILL_AMT5	30000.0	40311.400967	60797.155770	-81334.0	1763.00	18104.5	50190.0
BILL_AMT6	30000.0	38871.760400	59554.107537	-339603.0	1256.00	17071.0	49198.0
PAY_AMT1	30000.0	5663.580500	16563.280354	0.0	1000.00	2100.0	5006.0
PAY_AMT2	30000.0	5921.163500	23040.870402	0.0	833.00	2009.0	5000.0
PAY_AMT3	30000.0	5225.681500	17606.961470	0.0	390.00	1800.0	4505.0
PAY_AMT4	30000.0	4826.076867	15666.159744	0.0	296.00	1500.0	4013.0
PAY_AMT5	30000.0	4799.387633	15278.305679	0.0	252.50	1500.0	4031.0
PAY_AMT6	30000.0	5215.502567	17777.465775	0.0	117.75	1500.0	4000.0
default payment next month	30000.0	0.221200	0.415062	0.0	0.00	0.0	0.0



```
In [16]: bill_columns = [
    "BILL_AMT1", "BILL_AMT2", "BILL_AMT3",
    "BILL_AMT4", "BILL_AMT5", "BILL_AMT6"
]
```



```
for col in bill_columns:
    print(col, (df[col] < 0).sum())
```

BILL_AMT1 590
 BILL_AMT2 669
 BILL_AMT3 655
 BILL_AMT4 675
 BILL_AMT5 655
 BILL_AMT6 688

```
In [17]: df[df["BILL_AMT1"] < 0][
         ["BILL_AMT1", "PAY_AMT1", "PAY_0", "default payment next month"]
         ].head(20)
```

Out[17]:

	BILL_AMT1	PAY_AMT1	PAY_0	default payment next month
--	-----------	----------	-------	----------------------------

26	-109	0	1	1
68	-190	0	1	0
92	-2000	7555	-2	0
101	-81	223	-2	0
109	-103	0	1	0
149	-1100	0	1	0
173	-709	0	1	1
216	-946	0	1	1
319	-2	0	1	0
330	-179	1000	1	1
351	-14386	10118	1	0
365	-9	1529	1	0
391	-1312	38621	1	1
521	-58	33049	1	0
528	-8	0	1	0
546	-65	0	1	1
599	-475	10000	-1	1
653	-200	0	1	1
727	-14	2582	-2	0
791	-54	600	1	0

```
In [18]: for col in pay_columns:
         print(col, sorted(df[col].unique()))
```

```
PAY_0 [np.int64(-2), np.int64(-1), np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8)]
PAY_2 [np.int64(-2), np.int64(-1), np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8)]
PAY_3 [np.int64(-2), np.int64(-1), np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8)]
PAY_4 [np.int64(-2), np.int64(-1), np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8)]
PAY_5 [np.int64(-2), np.int64(-1), np.int64(0), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8)]
PAY_6 [np.int64(-2), np.int64(-1), np.int64(0), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8)]
```

```
In [19]: df.columns
```

```
Out[19]: Index(['ID', 'LIMIT_BAL', 'SEX', 'EDUCATION', 'MARRIAGE', 'AGE', 'PAY_0',
               'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6', 'BILL_AMT1', 'BILL_AMT2',
               'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6', 'PAY_AMT1',
               'PAY_AMT2', 'PAY_AMT3', 'PAY_AMT4', 'PAY_AMT5', 'PAY_AMT6',
               'default payment next month'],
              dtype='str')
```

```
In [20]: df_clean = df.copy()
```

```
In [1]: df_clean = df.copy()
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[1], line 1
----> 1 df_clean = df.copy()

NameError: name 'df' is not defined
```

```
In [2]: import pandas as pd
```

```
df = pd.read_excel("default of credit card clients.xls", header=1)
```

```
In [3]: df.shape
```

```
Out[3]: (30000, 25)
```

```
In [4]: df_clean = df.copy()
```

```
In [5]: df_clean["EDUCATION"].value_counts().sort_index()
```

```
Out[5]: EDUCATION
0         14
1      10585
2      14030
3       4917
4        123
5        280
6         51
Name: count, dtype: int64
```

```
In [6]: df_clean.groupby("EDUCATION")["default payment next month"].agg(["count", "mean"])
```

```
Out[6]:
```

	count	mean
EDUCATION		
0	14	0.000000
1	10585	0.192348
2	14030	0.237349
3	4917	0.251576
4	123	0.056911
5	280	0.064286
6	51	0.156863

```
In [7]: df_clean["MARRIAGE"].value_counts().sort_index()
```

```
Out[7]: MARRIAGE
0         54
1      13659
2      15964
3         323
Name: count, dtype: int64
```

```
In [8]: df_clean.groupby("MARRIAGE")["default payment next month"].agg(["count", "mean"])
```

```
Out[8]:
```

	count	mean
MARRIAGE		
0	54	0.092593
1	13659	0.234717
2	15964	0.209283
3	323	0.260062

```
In [9]: # Keep MARRIAGE as it is
```

```
In [10]: pay_columns = ["PAY_0", "PAY_2", "PAY_3", "PAY_4", "PAY_5", "PAY_6"]

for col in pay_columns:
    print("\n", col)
    print(df_clean[col].value_counts().sort_index())
```

```
PAY_0
PAY_0
-2      2759
-1      5686
0      14737
1       3688
2       2667
3        322
4         76
5         26
6         11
7          9
8         19
Name: count, dtype: int64
```

```
PAY_2
PAY_2
-2      3782
-1     6050
0     15730
1         28
2     3927
3      326
4       99
5       25
6       12
7       20
8         1
Name: count, dtype: int64
```

```
PAY_3
PAY_3
-2     4085
-1     5938
0     15764
1         4
2     3819
3     240
4       76
5       21
6       23
7       27
8         3
Name: count, dtype: int64
```

```
PAY_4
PAY_4
-2     4348
-1     5687
0     16455
1         2
2     3159
3     180
4       69
5       35
6         5
```

7	58
---	----

8	2
---	---

Name: count, dtype: int64

PAY_5

PAY_5

-2	4546
----	------

-1	5539
----	------

0	16947
---	-------

2	2626
---	------

3	178
---	-----

4	84
---	----

5	17
---	----

6	4
---	---

7	58
---	----

8	1
---	---

Name: count, dtype: int64

PAY_6

PAY_6

-2	4895
----	------

-1	5740
----	------

0	16286
---	-------

2	2766
---	------

3	184
---	-----

4	49
---	----

5	13
---	----

6	19
---	----

7	46
---	----

8	2
---	---

Name: count, dtype: int64

```
In [11]: df_clean.groupby("PAY_0")["default payment next month"].agg(["count", "mean"])
```

Out[11]:

	count	mean
PAY_0		
-2	2759	0.132294
-1	5686	0.167781
0	14737	0.128113
1	3688	0.339479
2	2667	0.691414
3	322	0.757764
4	76	0.684211
5	26	0.500000
6	11	0.545455
7	9	0.777778
8	19	0.578947

```
In [12]: bill_columns = [
            "BILL_AMT1", "BILL_AMT2", "BILL_AMT3",
            "BILL_AMT4", "BILL_AMT5", "BILL_AMT6"
        ]

        for col in bill_columns:
            print(col, "negative:", (df_clean[col] < 0).sum())
```

```
BILL_AMT1 negative: 590
BILL_AMT2 negative: 669
BILL_AMT3 negative: 655
BILL_AMT4 negative: 675
BILL_AMT5 negative: 655
BILL_AMT6 negative: 688
```

```
In [13]: df_clean[df_clean["BILL_AMT1"] < 0][
            ["BILL_AMT1", "PAY_0", "default payment next month"]
        ].groupby("PAY_0").agg(
            count=("BILL_AMT1", "count"),
            default_rate=("default payment next month", "mean")
        )
```

Out[13]:

	count	default_rate
PAY_0		
-2	209	0.157895
-1	4	0.250000
1	377	0.198939

In [14]: *# Define features and target*

```
X = df_clean.drop(columns=["default payment next month", "ID"])
y = df_clean["default payment next month"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

X shape: (30000, 23)

y shape: (30000,)

In [15]: **from** sklearn.model_selection **import** train_test_split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
print("Training set:", X_train.shape)
```

```
print("Testing set:", X_test.shape)
```

Training set: (24000, 23)

Testing set: (6000, 23)

In [16]: **from** sklearn.linear_model **import** LogisticRegression

```
log_model = LogisticRegression(max_iter=1000)
```

```
log_model.fit(X_train, y_train)
```

C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\linear_model_logistic.py:599: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=1):

STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).

You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

Out[16]:

▼ LogisticRegression ⓘ ?

- ▶ Parameters
- ▶ Fitted attributes

In [17]: **from** sklearn.preprocessing **import** StandardScaler

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
In [18]: log_model = LogisticRegression(max_iter=1000)

log_model.fit(X_train_scaled, y_train)
```

```
Out[18]:
```

▼ **LogisticRegression** ⓘ ?

- ▶ Parameters
- ▶ Fitted attributes

```
In [19]: y_pred = log_model.predict(X_test_scaled)

print(y_pred[:20])
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

```
In [20]: from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Accuracy: 0.8076666666666666

Confusion Matrix:

```
[[4528 145]
 [1009 318]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.82	0.97	0.89	4673
1	0.69	0.24	0.36	1327
accuracy			0.81	6000
macro avg	0.75	0.60	0.62	6000
weighted avg	0.79	0.81	0.77	6000

```
In [21]: y_prob = log_model.predict_proba(X_test_scaled)[: , 1]

print(y_prob[:20])
```



```
[0.10921496 0.11668539 0.22019774 0.09660511 0.02750998 0.20087738
 0.13159178 0.15358892 0.07595841 0.19481887 0.33498546 0.07853729
 0.07353147 0.1747379 0.2023213 0.4423843 0.01541936 0.35257735
 0.17901985 0.16696765]
```

In [2]: `from sklearn.linear_model import LogisticRegression`

```
balanced_model = LogisticRegression(
    max_iter=1000,
    class_weight="balanced"
)

balanced_model.fit(X_train_scaled, y_train)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[2], line 8
      4     max_iter=1000,
      5     class_weight="balanced"
      6 )
      7
----> 8 balanced_model.fit(X_train_scaled, y_train)

NameError: name 'X_train_scaled' is not defined
```

In [3]: `from sklearn.preprocessing import StandardScaler`

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[3], line 5
      1 from sklearn.preprocessing import StandardScaler
      2
      3 scaler = StandardScaler()
      4
----> 5 X_train_scaled = scaler.fit_transform(X_train)
      6 X_test_scaled = scaler.transform(X_test)

NameError: name 'X_train' is not defined
```

In [12]: `import pandas as pd`

```
df = pd.read_excel(
    "default of credit card clients.xls",
    header=1
)

df_clean = df.copy()
```

In [5]: `X = df_clean.drop(
 columns=["default payment next month", "ID"]
)`

```
y = df_clean["default payment next month"]
```

```
In [8]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
In [13]: print("Model trained successfully!")
print(balanced_model)
```

Model trained successfully!
LogisticRegression(class_weight='balanced', max_iter=1000)

```
In [14]: y_pred_balanced = balanced_model.predict(X_test_scaled)

print(y_pred_balanced[:20])
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[14], line 1
----> 1 y_pred_balanced = balanced_model.predict(X_test_scaled)
      2
      3 print(y_pred_balanced[:20])

NameError: name 'X_test_scaled' is not defined
```

```
In [15]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Scaling completed successfully!")
```

Scaling completed successfully!

```
In [16]: y_pred_balanced = balanced_model.predict(X_test_scaled)

print(y_pred_balanced[:20])
```

```

-----
NotFittedError                                Traceback (most recent call last)
Cell In[16], line 1
----> 1 y_pred_balanced = balanced_model.predict(X_test_scaled)
      2
      3 print(y_pred_balanced[:20])

File ~\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\linear_model_base.py:414, in LinearClassifierMixin.predict(self, X)
    412 check_same_namespace(X, self, attribute="coef_", method="predict")
    413 xp, _, device_ = get_namespace_and_device(X)
--> 414 scores = self.decision_function(X)
    415 if len(scores.shape) == 1:
    416     indices = xp.astype(scores > 0, indexing_dtype(xp))

File ~\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\linear_model_base.py:385, in LinearClassifierMixin.decision_function(self, X)
    366 def decision_function(self, X):
    367     """
    368     Predict confidence scores for samples.
    369
    (...) 383         this class would be predicted.
    384     """
--> 385     check_is_fitted(self)
    386     xp, _ = get_namespace(X)
    387     check_same_namespace(X, self, attribute="coef_", method="decision_function")

File ~\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\utils\validation.py:1718, in check_is_fitted(estimator, attributes, msg, all_or_any)
    1715     return
    1717 if not _is_fitted(estimator, attributes, all_or_any):
-> 1718     raise NotFittedError(msg % {"name": type(estimator).__name__})

NotFittedError: This LogisticRegression instance is not fitted yet. Call 'fit' with
appropriate arguments before using this estimator.

```

```

In [17]: balanced_model.fit(X_train_scaled, y_train)

        print("Model fitted successfully!")

```

Model fitted successfully!

```

In [18]: y_pred_balanced = balanced_model.predict(X_test_scaled)

        print(y_pred_balanced[:20])

```

[0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 1 0 0]

```

In [19]: from sklearn.metrics import confusion_matrix

        cm = confusion_matrix(y_test, y_pred_balanced)

        print(cm)

```

[[3255 1418]
 [504 823]]

```
In [20]: from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_balanced))
```

	precision	recall	f1-score	support
0	0.87	0.70	0.77	4673
1	0.37	0.62	0.46	1327
accuracy			0.68	6000
macro avg	0.62	0.66	0.62	6000
weighted avg	0.76	0.68	0.70	6000

```
In [21]: from sklearn.metrics import roc_auc_score

y_prob_balanced = balanced_model.predict_proba(X_test_scaled)[: , 1]

auc = roc_auc_score(y_test, y_prob_balanced)

print("ROC-AUC:", auc)
```

ROC-AUC: 0.7081149369197677

```
In [22]: threshold = 0.40

y_pred_040 = (y_prob_balanced >= threshold).astype(int)

print(y_pred_040[:20])
```

[0 0 1 0 0 1 0 0 0 1 1 0 0 1 1 1 0 1 1 1]

```
In [23]: from sklearn.metrics import confusion_matrix

cm_040 = confusion_matrix(y_test, y_pred_040)

print(cm_040)
```

```
[[1981 2692]
 [ 298 1029]]
```

```
In [24]: from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_040))
```

	precision	recall	f1-score	support
0	0.87	0.42	0.57	4673
1	0.28	0.78	0.41	1327
accuracy			0.50	6000
macro avg	0.57	0.60	0.49	6000
weighted avg	0.74	0.50	0.53	6000

```
In [25]: from sklearn.metrics import precision_score, recall_score, f1_score
```

```

thresholds = [0.30, 0.40, 0.50, 0.60, 0.70]

for threshold in thresholds:
    y_pred = (y_prob_balanced >= threshold).astype(int)

    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print(
        "Threshold:", threshold,
        "| Precision:", round(precision, 3),
        "| Recall:", round(recall, 3),
        "| F1:", round(f1, 3)
    )

```

```

Threshold: 0.3 | Precision: 0.244 | Recall: 0.888 | F1: 0.383
Threshold: 0.4 | Precision: 0.277 | Recall: 0.775 | F1: 0.408
Threshold: 0.5 | Precision: 0.367 | Recall: 0.62 | F1: 0.461
Threshold: 0.6 | Precision: 0.558 | Recall: 0.449 | F1: 0.497
Threshold: 0.7 | Precision: 0.637 | Recall: 0.347 | F1: 0.449

```

```

In [26]: import numpy as np
        from sklearn.metrics import f1_score

        thresholds = np.arange(0.10, 0.91, 0.01)

        best_threshold = 0
        best_f1 = 0

        for threshold in thresholds:
            y_pred = (y_prob_balanced >= threshold).astype(int)
            f1 = f1_score(y_test, y_pred)

            if f1 > best_f1:
                best_f1 = f1
                best_threshold = threshold

        print("Best threshold:", round(best_threshold, 2))
        print("Best F1-score:", round(best_f1, 3))

```

```

Best threshold: 0.6
Best F1-score: 0.497

```

```

In [27]: final_threshold = 0.60

        y_pred_final = (y_prob_balanced >= final_threshold).astype(int)

        print(y_pred_final[:20])

```

```
[0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 1 0 0]
```

```

In [28]: from sklearn.metrics import confusion_matrix

        cm_final = confusion_matrix(y_test, y_pred_final)

```

```
print(cm_final)
```

```
[[4200  473]
 [ 731  596]]
```

```
In [29]: from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_final))
```

	precision	recall	f1-score	support
0	0.85	0.90	0.87	4673
1	0.56	0.45	0.50	1327
accuracy			0.80	6000
macro avg	0.70	0.67	0.69	6000
weighted avg	0.79	0.80	0.79	6000

```
In [30]: from sklearn.metrics import roc_auc_score

y_prob_original = log_model.predict_proba(X_test_scaled)[: , 1]

auc_original = roc_auc_score(y_test, y_prob_original)

print("Original Logistic Regression ROC-AUC:", auc_original)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[30], line 3
      1 from sklearn.metrics import roc_auc_score
      2
----> 3 y_prob_original = log_model.predict_proba(X_test_scaled)[: , 1]
      4
      5 auc_original = roc_auc_score(y_test, y_prob_original)
      6

NameError: name 'log_model' is not defined
```

```
In [31]: from sklearn.linear_model import LogisticRegression

log_model = LogisticRegression(max_iter=1000)

log_model.fit(X_train_scaled, y_train)

print("Original model fitted successfully!")
```

Original model fitted successfully!

```
In [32]: from sklearn.metrics import roc_auc_score

y_prob_original = log_model.predict_proba(X_test_scaled)[: , 1]

auc_original = roc_auc_score(y_test, y_prob_original)

print("Original Logistic Regression ROC-AUC:", auc_original)
```

Original Logistic Regression ROC-AUC: 0.7076355036089734

```
In [33]: from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight="balanced"
)

rf_model.fit(X_train, y_train)

print("Random Forest trained successfully!")
```

Random Forest trained successfully!

```
In [34]: y_pred_rf = rf_model.predict(X_test)

print(y_pred_rf[:20])
```

[0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 1 0 0 0 0]

```
In [35]: from sklearn.metrics import confusion_matrix

cm_rf = confusion_matrix(y_test, y_pred_rf)

print(cm_rf)
```

```
[[4153  520]
 [ 679  648]]
```

```
In [36]: from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_rf))
```

	precision	recall	f1-score	support
0	0.86	0.89	0.87	4673
1	0.55	0.49	0.52	1327
accuracy			0.80	6000
macro avg	0.71	0.69	0.70	6000
weighted avg	0.79	0.80	0.80	6000

```
In [37]: from sklearn.metrics import roc_auc_score

y_prob_rf = rf_model.predict_proba(X_test)[: , 1]

auc_rf = roc_auc_score(y_test, y_prob_rf)

print("Random Forest ROC-AUC:", auc_rf)
```

Random Forest ROC-AUC: 0.7589071307198385

```
In [38]: import numpy as np

from sklearn.metrics import f1_score
```

```

thresholds = np.arange(0.10, 0.91, 0.01)

best_threshold_rf = 0
best_f1_rf = 0

for threshold in thresholds:
    y_pred = (y_prob_rf >= threshold).astype(int)
    f1 = f1_score(y_test, y_pred)

    if f1 > best_f1_rf:
        best_f1_rf = f1
        best_threshold_rf = threshold

print("Best Random Forest threshold:", round(best_threshold_rf, 2))
print("Best Random Forest F1-score:", round(best_f1_rf, 3))

```

Best Random Forest threshold: 0.43

Best Random Forest F1-score: 0.526

```

In [39]: final_threshold_rf = 0.43

y_pred_rf_final = (y_prob_rf >= final_threshold_rf).astype(int)

print(y_pred_rf_final[:20])

```

[0 0 0 0 0 1 0 0 0 0 0 1 1 0 0 1 0 1 0 0]

```

In [40]: from sklearn.metrics import confusion_matrix

cm_rf_final = confusion_matrix(y_test, y_pred_rf_final)

print(cm_rf_final)

```

```

[[3935  738]
 [ 591  736]]

```

```

In [41]: from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_rf_final))

```

	precision	recall	f1-score	support
0	0.87	0.84	0.86	4673
1	0.50	0.55	0.53	1327
accuracy			0.78	6000
macro avg	0.68	0.70	0.69	6000
weighted avg	0.79	0.78	0.78	6000

```

In [42]: import pandas as pd

feature_importance = pd.Series(
    rf_model.feature_importances_,
    index=X_train.columns
).sort_values(ascending=False)

```



```
print(feature_importance.head(10))
```

```
PAY_0      0.100603
LIMIT_BAL  0.064435
BILL_AMT1   0.062294
AGE         0.062274
BILL_AMT2   0.053145
PAY_AMT1    0.051128
PAY_AMT2    0.051017
BILL_AMT3   0.050851
BILL_AMT4   0.049106
PAY_AMT3    0.048623
dtype: float64
```

```
In [43]: comparison = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Random Forest"
    ],
    "Accuracy": [
        0.80,
        0.78
    ],
    "Default Precision": [
        0.56,
        0.50
    ],
    "Default Recall": [
        0.45,
        0.55
    ],
    "Default F1": [
        0.50,
        0.53
    ],
    "ROC-AUC": [
        0.7081,
        0.7589
    ]
})

print(comparison)
```

	Model	Accuracy	Default Precision	Default Recall	\
0	Logistic Regression	0.80	0.56	0.45	
1	Random Forest	0.78	0.50	0.55	
	Default F1	ROC-AUC			
0	0.50	0.7081			
1	0.53	0.7589			

```
In [44]: import matplotlib.pyplot as plt

metrics = ["Accuracy", "Default Precision", "Default Recall", "Default F1", "ROC-AU

lr_values = [0.80, 0.56, 0.45, 0.50, 0.7081]
```

```

rf_values = [0.78, 0.50, 0.55, 0.53, 0.7589]

x = range(len(metrics))

plt.figure(figsize=(10, 5))

plt.plot(x, lr_values, marker="o", label="Logistic Regression")
plt.plot(x, rf_values, marker="o", label="Random Forest")

plt.xticks(x, metrics)
plt.ylabel("Score")
plt.title("Model Performance Comparison")
plt.legend()
plt.grid(True)

plt.show()

```

```

-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[44], line 1
----> 1 import matplotlib.pyplot as plt
      2
      3 metrics = ["Accuracy", "Default Precision", "Default Recall", "Default F1",
"ROC-AUC"]
      4

ModuleNotFoundError: No module named 'matplotlib'

```

```

In [45]: import joblib

joblib.dump(rf_model, "credit_risk_random_forest.pkl")

print("Final Random Forest model saved successfully!")

```

Final Random Forest model saved successfully!

```

In [46]: !pip install matplotlib

```

```

'pip' is not recognized as an internal or external command,
operable program or batch file.

```

```

In [48]: import sys
!{sys.executable} -m pip install matplotlib

```

Requirement already satisfied: matplotlib in C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.11.2)

Requirement already satisfied: contourpy>=1.0.1 in C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.4.0)

Requirement already satisfied: cyclor>=0.10 in C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.28.2 in C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (4.66.1)

Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.5.1)

Requirement already satisfied: numpy>=1.25 in C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.4.6)

Requirement already satisfied: packaging>=20.0 in C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (26.2)

Requirement already satisfied: pillow>=9 in C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (12.3.0)

Requirement already satisfied: pyparsing>=3 in C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (3.3.3)

Requirement already satisfied: python-dateutil>=2.7 in C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in C:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

[notice] A new release of pip is available: 26.1.2 -> 26.2.1

[notice] To update, run: C:\Users\Admin\AppData\Local\Programs\Python\Python314\python.exe -m pip install --upgrade pip

```
In [49]: import matplotlib.pyplot as plt

metrics = ["Accuracy", "Default Precision", "Default Recall", "Default F1", "ROC-AU

lr_values = [0.80, 0.56, 0.45, 0.50, 0.7081]
rf_values = [0.78, 0.50, 0.55, 0.53, 0.7589]

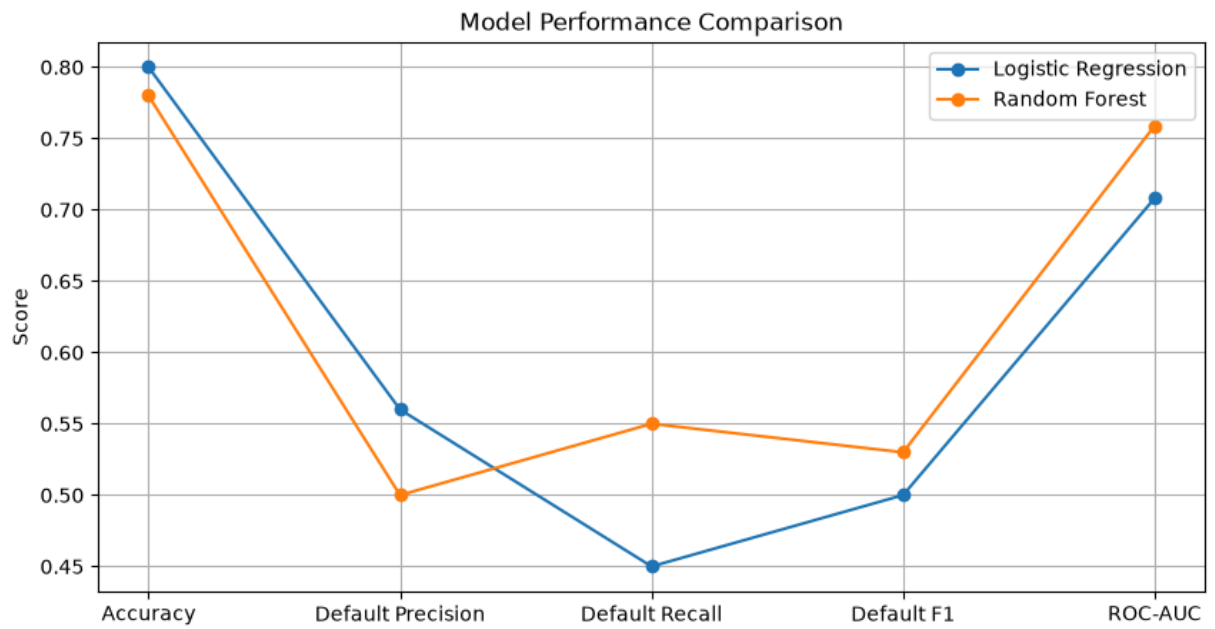
x = range(len(metrics))

plt.figure(figsize=(10, 5))

plt.plot(x, lr_values, marker="o", label="Logistic Regression")
plt.plot(x, rf_values, marker="o", label="Random Forest")

plt.xticks(x, metrics)
plt.ylabel("Score")
plt.title("Model Performance Comparison")
plt.legend()
plt.grid(True)

plt.show()
```

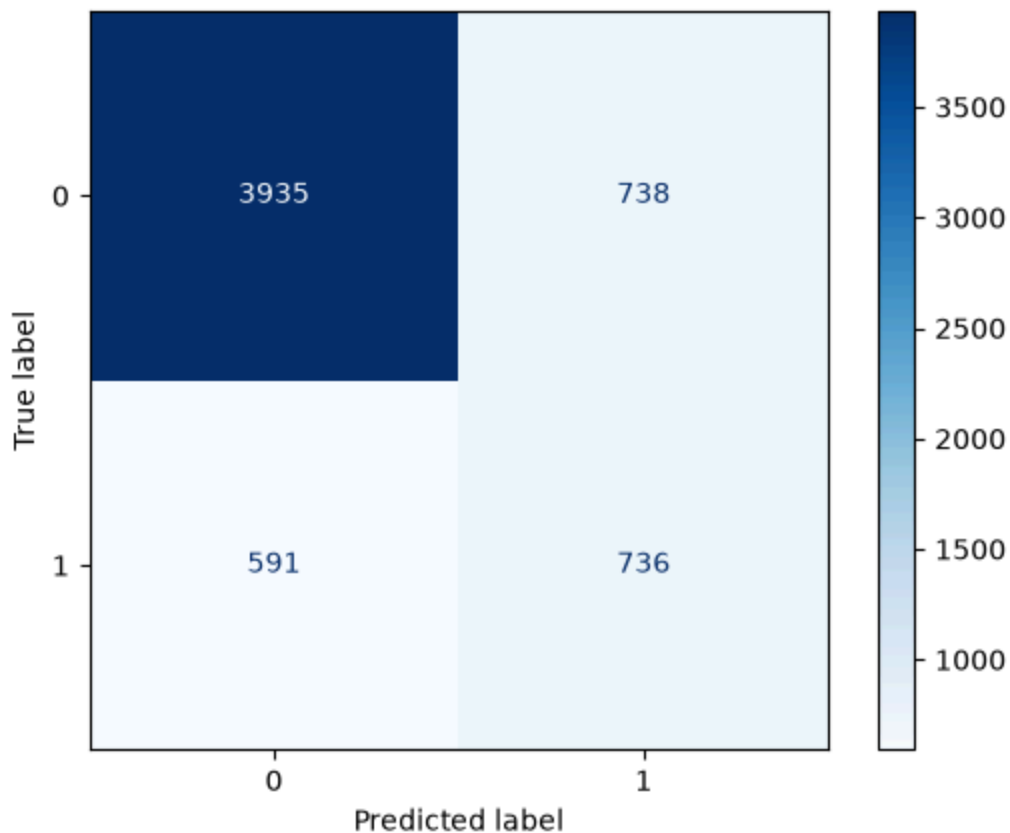


```
In [50]: import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred_rf_final,
    cmap="Blues"
)

plt.title("Random Forest Confusion Matrix (Threshold = 0.43)")
plt.show()
```

Random Forest Confusion Matrix (Threshold = 0.43)



```
In [51]: from sklearn.metrics import RocCurveDisplay
import matplotlib.pyplot as plt

RocCurveDisplay.from_predictions(
    y_test,
    y_prob_balanced,
    name="Logistic Regression"
)

RocCurveDisplay.from_predictions(
    y_test,
    y_prob_rf,
    name="Random Forest"
)

plt.title("ROC Curve: Logistic Regression vs Random Forest")
plt.show()
```

Cell In[51], line 1

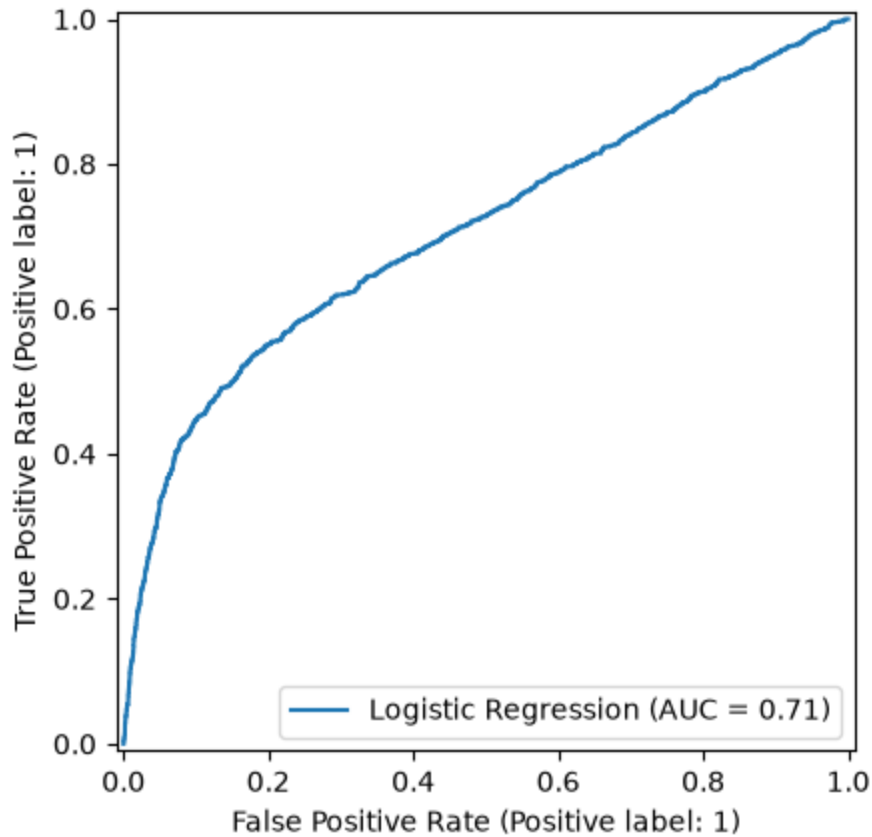
```
from sklearn.metrics import RocCurveDisplay
^
```

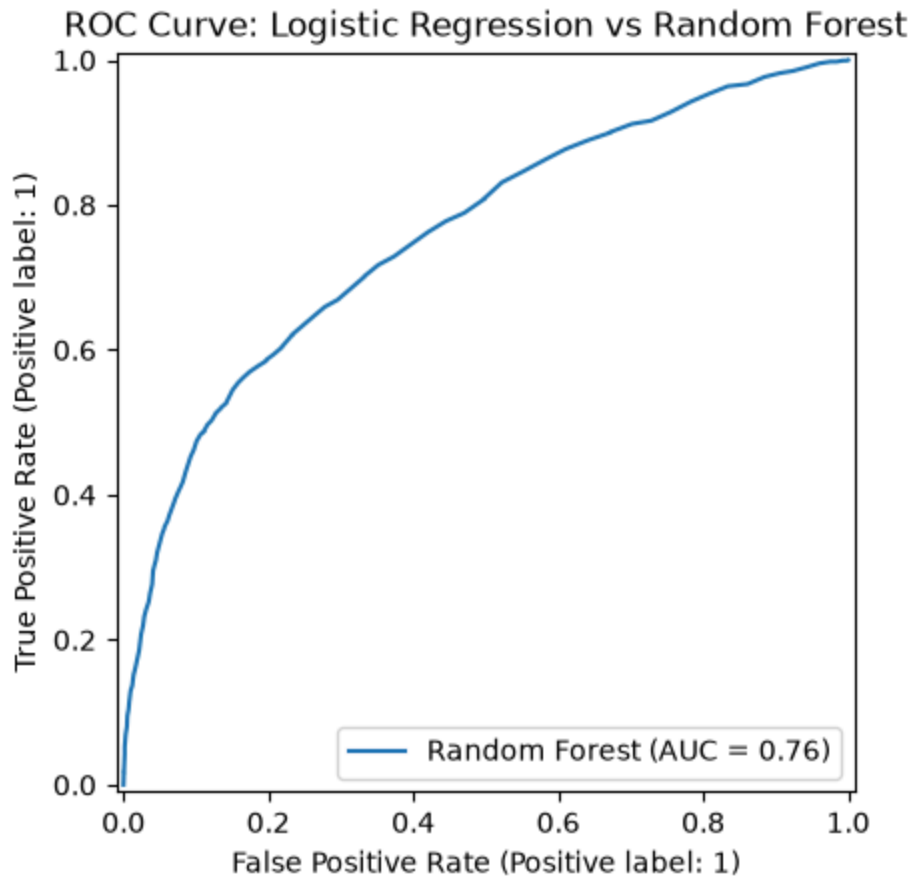
SyntaxError: invalid syntax

```
In [52]: from sklearn.metrics import RocCurveDisplay
import matplotlib.pyplot as plt

RocCurveDisplay.from_predictions(
    y_test,
```

```
y_prob_balanced,  
name="Logistic Regression"  
)  
  
RocCurveDisplay.from_predictions(  
    y_test,  
    y_prob_rf,  
    name="Random Forest"  
)  
  
plt.title("ROC Curve: Logistic Regression vs Random Forest")  
plt.show()
```





```
In [53]: from sklearn.metrics import precision_score, recall_score, f1_score
```

```
thresholds = [0.20, 0.25, 0.30, 0.35, 0.40, 0.43, 0.45, 0.50]
```

```
for threshold in thresholds:
    y_pred = (y_prob_rf >= threshold).astype(int)

    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print(
        f"Threshold: {threshold:.2f} | "
        f"Precision: {precision:.3f} | "
        f"Recall: {recall:.3f} | "
        f"F1: {f1:.3f}"
    )
```

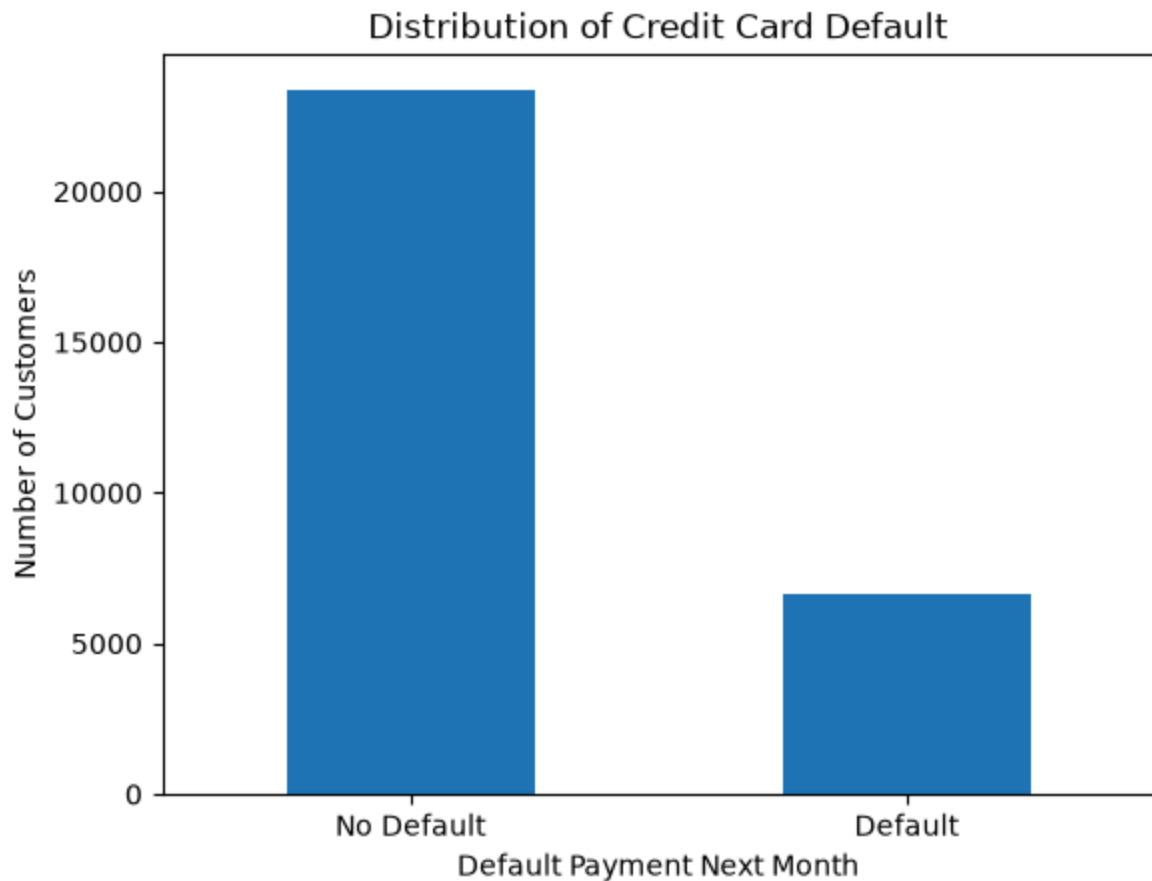
```
Threshold: 0.20 | Precision: 0.296 | Recall: 0.863 | F1: 0.441
Threshold: 0.25 | Precision: 0.332 | Recall: 0.778 | F1: 0.465
Threshold: 0.30 | Precision: 0.375 | Recall: 0.701 | F1: 0.488
Threshold: 0.35 | Precision: 0.422 | Recall: 0.633 | F1: 0.506
Threshold: 0.40 | Precision: 0.470 | Recall: 0.577 | F1: 0.518
Threshold: 0.43 | Precision: 0.499 | Recall: 0.555 | F1: 0.526
Threshold: 0.45 | Precision: 0.510 | Recall: 0.537 | F1: 0.523
Threshold: 0.50 | Precision: 0.550 | Recall: 0.497 | F1: 0.522
```

```
In [54]: import matplotlib.pyplot as plt
```

```
df["default payment next month"].value_counts().sort_index().plot(
    kind="bar"
)

plt.title("Distribution of Credit Card Default")
plt.xlabel("Default Payment Next Month")
plt.ylabel("Number of Customers")
plt.xticks([0, 1], ["No Default", "Default"], rotation=0)

plt.show()
```

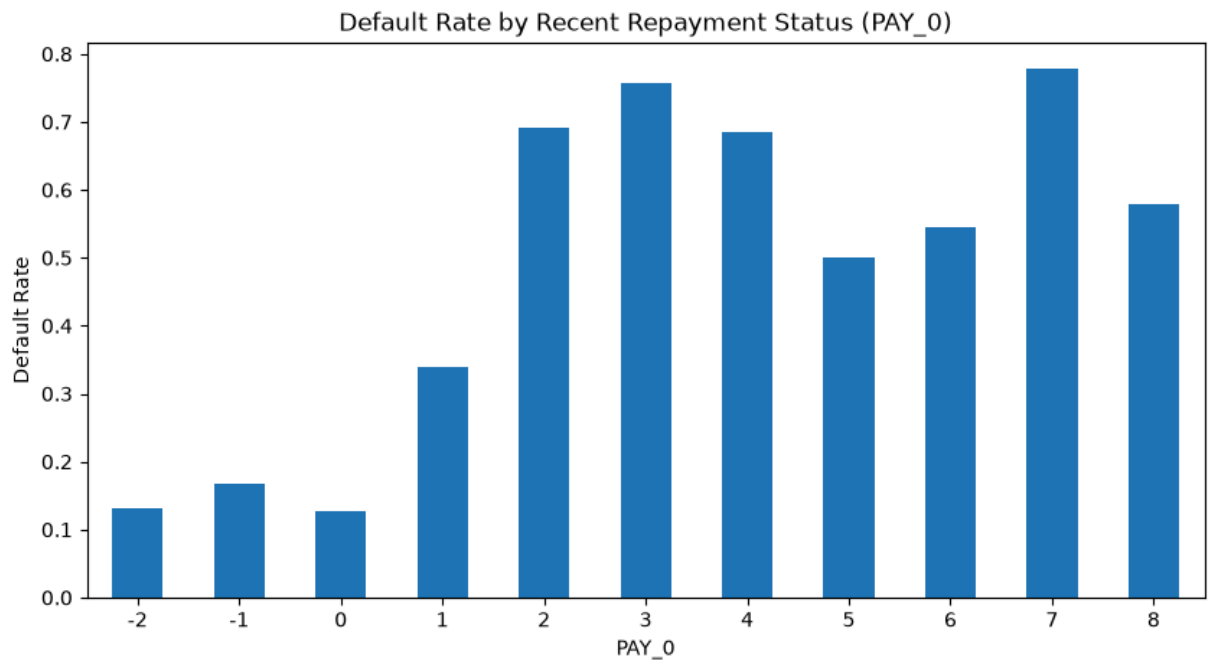


```
In [55]: default_rate_pay0 = df_clean.groupby("PAY_0")["default payment next month"].mean()

default_rate_pay0.plot(kind="bar", figsize=(10, 5))

plt.title("Default Rate by Recent Repayment Status (PAY_0)")
plt.xlabel("PAY_0")
plt.ylabel("Default Rate")
plt.xticks(rotation=0)

plt.show()
```

In []: